Embedded Engineer take-home assignment

Estimated effort: 6–10 hours. Please timebox it — we care about design judgment and clarity of trade-off reasoning more than a fully polished build.

Context

Paw is a LoRa-based pet tracking collar. The collar has no SIM/cellular connection by design. It talks directly to the owner's phone over LoRa point-to-point (P2P) when nearby, and to a home LoRaWAN gateway for wider village-range coverage. This task asks you to design and partially build the collar's hardware and firmware, plus the bridge that gets a location fix into a mobile app over MQTT.

Part 1 — PCB design

Design a schematic for the collar's main board. A full manufacturable layout is not required — we're evaluating your component selection, RF reasoning, and documentation.

- Radio + MCU: RAK3172 module (STM32WLE5-based) or the bare STM32WLE5CCU6 SoC — your choice, but justify it.
- GNSS: a low-power GPS module (e.g. u-blox MAX-M10S class).
- Motion sensing: a low-power accelerometer (e.g. LIS2DW12 class) for wake-on-motion.
- Power: single-cell LiPo with a charge management IC (e.g. TP4056 class), plus a regulator sized for the radio's peak TX current.
- RF: 868 MHz antenna matching network. Show your matching topology and explain your placement decisions relative to the ground plane and battery.

Deliverables:

#	Deliverable
1	Schematic (KiCad preferred, or a clearly labelled hand-drawn/PDF schematic if tooling is a constraint)
2	Bill of materials with part numbers and approximate unit cost
3	One-page write-up: antenna placement rationale, power budget estimate (sleep vs. TX vs. GPS-on), and what you'd change for a production run at 1,000 units

Part 2 — Embedded C firmware

Implement collar firmware (bare-metal C, or using RAKwireless RUI3 APIs / STM32 HAL — your choice) that:

- Wakes on an accelerometer motion interrupt, otherwise stays in deep sleep.
- On wake, acquires a GPS fix (simulate the GPS response if you don't have hardware — a mocked NMEA/UBX parser is fine).
- Packs {device_id, lat, lon, timestamp, battery_mv} into a compact binary payload and transmits it via LoRa P2P.
- Returns to sleep, with a maximum fix interval even without motion (e.g. one fix every 30 minutes) so the collar isn't silent for hours.

Deliverables:

#	Deliverable
1	Firmware source (can target real hardware, a simulator, or be structured with hardware-abstraction stubs — say which)
2	A short state machine diagram (sleep / wake / fix / transmit / back to sleep)
3	A paragraph explaining your power-optimization choices and estimated battery life at a stated fix rate

Part 3 — MQTT bridge to the mobile app

Build the bridge that takes a LoRa packet received at the gateway and gets it in front of the mobile app in near real time.

- A bridge process (e.g. on an ESP32, Raspberry Pi, or a simulated script reading mock LoRa packets from a file/serial port) that decodes the payload and publishes it to an MQTT broker.
- Define your topic structure and payload schema, for example:

```
topic:   paw/collar/{device_id}/location
payload: {
  "lat": 52.4104,
  "lon": 12.9738,
  "ts": 1735689600,
  "battery_mv": 3850,
  "source": "gateway"
}
```

- QoS choice: state which QoS level (0/1/2) you'd use for location updates and why.
- A minimal mobile-app-side subscriber: either working code (Flutter/React Native/Android/iOS, your choice) or clear pseudocode showing how the app subscribes and updates the map on message receipt.

Deliverables:

#	Deliverable
1	Bridge source code
2	MQTT topic/payload schema documentation (can be the snippet above, extended if you add fields)
3	App-side subscription code or pseudocode, plus your QoS and retained-message rationale

Submission

- A git repository (zip or link) containing all three parts, with a top-level README explaining what's real vs. simulated/stubbed and how to run each part.
- Please note any assumptions you made and anything you'd do differently with more time.

Evaluation criteria

- Correctness and clarity of the RF/antenna and power-budget reasoning
- Code quality and readability in both the firmware and the bridge
- Sound low-power embedded design choices (sleep strategy, wake sources, duty cycling)
- MQTT/protocol design judgment (topic structure, payload shape, QoS choice)
- Documentation clarity — can another engineer pick this up and understand your decisions